MACHINE LEARNING WITH SCIKIT-LEARN

Final Project: Customer Churn Prediction

08

One real business problem, every skill from the course — EDA to pipeline to honest evaluation to a 5-minute story.

Master's Degree Program · Hands-on with Google Colab

AGENDA

Today's agenda

01 **The business problem**
why churn is money

02 **The dataset**
Telco: 7,043 customers, 19 columns

03 **The 8 project steps**
= the whole course, in order

04 **Real results to beat**
baseline & model benchmarks

05 **Error analysis & insight**
what drives churn

06 **Presenting an ML project**
the 5-minute structure

07 **Rubric & deliverables**
exactly how you're graded

08 **After this course**
XGBoost · deep learning · MLOps

THE PROBLEM

Churn: keeping a customer is 5–25× cheaper than finding one

The question

Which customers will leave us next month?

— before they leave, while a discount or a phone call can still save them.

The data

7,043 telecom customers
19 columns: contract, services, tenure, charges
Target: Churn (Yes/No) — 26.5% positive

The real deliverable

Not a model.

A ranked list of at-risk customers + the story of WHY — that a retention team can act on Monday.

This is supervised classification — you now own every tool for it.

DATASET

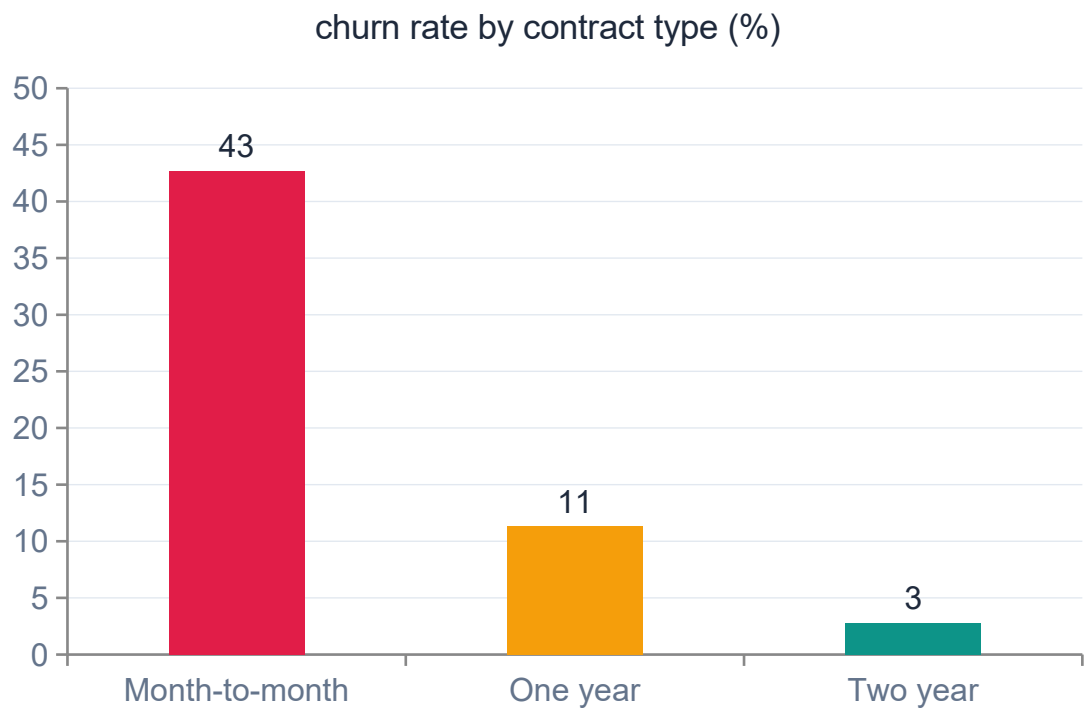
Telco churn — 7,043 × 19

column group	columns
target	Churn (Yes/No) — 26.5% Yes
account	tenure (months), Contract, PaperlessBilling, PaymentMethod
services (0/1)	PhoneService, MultipleLines, OnlineSecurity, … StreamingMovies
internet	InternetService: DSL / Fiber optic / No
charges	MonthlyCharges ($), TotalCharges ($) — ⚠ dtype trap
id	customerID — drop it (memorization risk)

```
url = "https://raw.githubusercontent.com/IBM/
        telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(url)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")  # 11
blanks → NaN
```

- ⚠ **TotalCharges trap**
- stored as text — 11 cells are " "
- to_numeric(errors="coerce") → NaN → imputer handles it (S02+S03!)
- **26.5% positive = mildly imbalanced**
- accuracy will lie a bit → AUC + recall (S04)

What the data already screams



- **Contract is destiny: 43% vs 3%**
- **Fiber customers churn 41.9%**
- vs DSL 19.0%, no internet 7.4% — pricey+flaky?
- **Churners: tenure 18 vs stayers 38 months**
- the first year is the danger zone
- **Churners pay more: $74.4 vs $61.3 / month**
- **Every one of these becomes a slide in YOUR presentation**

THE METHOD

The project IS the course

1
EDA & clean
S02 skills

→

2
Split FIRST
S03 golden rule

→

3
Pipeline
S03 ColumnTransformer

→

4
Baseline + metrics
S04 Dummy/AUC/recall

5
Model zoo (CV)
S05–06 compare families

→

6
Tune the finalist
S07 RandomizedSearch

→

7
Error analysis
S07 drivers & misses

→

8
Test ONCE + present
S04 + today

Starter notebook `08.final_project_starter.ipynb` has this exact skeleton with TODO cells — you fill the thinking.

STEPS 3–5

Benchmarks from my run (stratified 5-fold CV)

model	CV acc	CV ROC-AUC
Dummy ("nobody churns")	0.735	0.500
RandomForest	0.784	0.818
HistGradientBoosting	0.793	0.838
LogisticRegression	0.803	0.846

- **Read it like S04 taught you**
- accuracy 0.735 is FREE — ignore it forever
- AUC is the honest ladder: 0.500 → 0.846
- **Plot twist: plain LogisticRegression wins**
- clean features + readable story beats the forest here
- your job: reproduce, tune (S07), maybe beat it with XGBoost

STEP 6

The threshold is a BUSINESS decision

threshold (LR, test)	recall	precision	reads as
0.50 (default)	0.56	0.66	balanced
0.65	0.30	0.75	few, safe calls
0.35	0.71	0.54	catch more, waste offers

- **Retention offers cost money**
- a 300฿ coupon × 500 customers = 150,000฿
- if you can only AFFORD to call 100 customers →
- rank by predict_proba, take the top 100 (AUC thinking!)
- **threshold moves along the ROC curve from S04**

Report this way

"At threshold 0.5 we catch 56% of all churners; 66% of our calls hit real churners. Lowering to 0.35 catches 71% but 46% of calls are wasted.

Budget says 100 calls → top-100 list catches X%."

THAT is a decision-ready sentence.

STEP 7

What drives churn (LR weights, scaled)

feature	weight	reads as
tenure	−1.23	every extra month anchors customers
Contract: Two-year	−0.76	locked-in = safe
InternetService: Fiber	+0.64	fiber = churn rocket
Contract: Month-to-month	+0.59	free to leave, free to leave…
InternetService: DSL	−0.64	older, loyal crowd

- **Insight slide for your presentation:**
- "New month-to-month fiber customers are our flight risk — call them in month 3, not month 11."
- **Cross-check with EDA (slide 5): the model and the eyes agree · bonus: permutation importance as a second opinion**

Error analysis ≠ vibes

Look at the top misses:

· churners with LOW predicted risk (missed danger — why?)
· stayers with HIGH risk (future churn? save them proactively?)

find_common_traits(top_misses)
→ the insight slide writes itself.

STEP 8

The final word: test set, once

0.842

test ROC-AUC
(CV said 0.846 — honest)

0.56

recall @ 0.5
of all true churners

0.806

accuracy
(vs Dummy 0.735)

CV 0.846 → test 0.842: a small, honest gap. THIS sentence belongs in your report — the gap is the difference between a student and a data scientist.

COMMUNICATE

The 5-minute presentation skeleton

min	slide	content
0:00	Problem	one sentence + the business stake
0:30	Data & method	1 EDA chart + your pipeline diagram (no code dumps)
1:30	Results	CV vs test, right metric, baseline next to it
2:30	Insight	the drivers chart + one decision-ready sentence
3:30	Limits & next	what could break it (leak? drift?), what you'd try next
4:30	Ask	what you need from the audience

Never: code screenshots, 12 decimal places, "we tried 9 models" (nobody cares) — sell the DECISION.

GRADING

Rubric — 40% of the course

dimension	weight	full marks	zero marks
Clean pipeline	25%	ColumnTransformer+Pipeline, split-first	manual transforms / leakage
Honest evaluation	25%	baseline, right metric, CV vs test gap reported	test-set peeking
Error analysis & insight	20%	drivers + misses + decision sentence	score dump only
Presentation	15%	5-min skeleton, business language	code walkthrough
Reproducibility	15%	runs top-to-bottom, seeds set, joblib saved	random notebook order

No points for the highest AUC. Points for the cleanest, most honest, most actionable project.

DELIVERABLES

What you hand in

① Notebook

The completed starter notebook: all 8 steps, your own EDA & error analysis, markdown commentary in Thai or English.

② Model artifact

churn_pipeline.joblib — the fitted Pipeline (+ the one cell that loads & predicts on a raw row).

③ 5-minute talk

4–6 slides using today's skeleton. Present in class, English or Thai.

▸ **AI assistants: fully allowed — cite your prompts in a final cell, and be ready to explain every line (S00 policy)**

WARNING

The five classic project fails

fail	symptom	prevention
Peeked at the test set (tuned on it)	test = train in disguise	CV for all decisions
Reported accuracy only (26.5% positive!)	"73.5% accurate" = free	AUC + recall + baseline
Left TotalCharges as text	imputer silently errors / garbage	to_numeric(coerce) first
9 models, zero error analysis	a zoo, not a project	insight beats leaderboard
Reads like code, not a story	nobody can act on it	5-minute skeleton

FOR THE AMBITIOUS

Extension menu (bonus bragging rights)

XGBoost / LightGBM

%pip install xgboost
swap into the SAME pipeline
—
beat my 0.846 CV AUC?

Cost-based threshold

false negative costs 3,000฿,
false positive costs 300฿ —
pick the profit-maximizing
threshold.

Calibration

does "70% risk" mean 70%?
CalibratedClassifierCV —
probabilities you can trust.

Explain one customer

pick 1 test customer,
walk through WHY the
model
says they'll churn.

THE ROAD AHEAD

After this course

Deeper tabular ML

XGBoost · LightGBM · CatBoost
feature engineering at scale
time-series validation

Deep learning

fastai / PyTorch
(images, text, audio)
start with the “Is it a bird?” notebook

Production / MLOps

model serving & monitoring
drift, retraining, champion-challenger
MLflow, Docker, cloud

Everything you learned transfers: split honestly, measure honestly, explain simply.

LAB 08 · DO IT IN GOOGLE COLAB

โปรเจ็คจบคอร์ส — ทำนายลูกค้าที่จะยกเลิก (Churn)

YOUR MISSION (โจทย์ภาษาไทยใน Colab)

- EDA: อัตรา churn รวม + แยกตาม contract / internet / tenure — พาดหัวข่าว 3 ข้อ
- แบ่ง 80/20 stratify ก่อน แล้วสร้าง pipeline (จัดการ TotalCharges + one-hot 16 คอลัมน์)
- Baseline Dummy → เทียบ ≥3 โมเดลด้วย stratified CV (AUC เป็นหลัก)
- Tune โมเดลที่ดีที่สุด (RandomizedSearchCV) แล้ววัด test ครั้งเดียว — เทียบกับ CV
- Error analysis + top drivers → ประโยค decision-ready 1 ประโยค แล้วส่ง notebook + joblib + สไลด์ 5 นาที

Learning goal

Deliver an end-to-end churn predictor: clean pipeline, honest evaluation, business-ready insight — graded by the rubric.

Vibe coding corner

ลองถาม AI: "ช่วยรีวิวโค้ดโปรเจ็คฉันเหมือน senior data scientist — หา leakage, metric ผิด, และจุดที่ต้อง error analysis เพิ่ม" — ถือเป็น code review รอบแรกก่อนส่งจริง

KEY TAKEAWAYS

1. The project IS the course: EDA → split-first pipelines → baseline → CV comparison → tuning → error analysis → one honest test.
2. Churn results to beat: CV AUC 0.846 / test 0.842 with plain LogisticRegression — insight (tenure, contract, fiber) matters as much as the score.
3. The threshold is a business decision — recall vs precision is a budget conversation, not a math one.
4. Grading rewards honesty and actionability, never leaderboard scores.
5. You now have the full classical-ML toolkit — go build, measure honestly, and explain simply.